Theory and Algorithms for In-Context Learning

Weight-Space and Activation-Space Perspectives

Jy-yong Sohn

Department of Applied Statistics · Yonsei University

itml.yonsei.ac.kr

My lab · ITML @ Yonsei · Theory → Algorithms

Mathematical tools (information theory, optimization, learning theory) for the **full life cycle of foundation models** — explain what the model does, then design better algorithms

Pillar 1 · Pre-training

What representation does the model learn?

- Cross-modal feature heterogeneity in sparse autoencoders **NeurIPS'26**
- Similarities of embeddings in contrastive learning **ICML'25**
- Preventing class collapse in supervised contrastive learning **AISTATS'25**

Ongoing: multi-modal embedding trained on missing modalities

Pillar 2 · Post-training

How to adapt it without losing what it knows?

- **Fine-tuning without forgetting ICL ICML'26**
- Representational shifts in continual learning **ICML'25**
- Compositional reasoning in CLIP via text reconstruction **NeurIPS'25**

Ongoing: RL with verifiable rewards

Pillar 3 · Inference

How to make reasoning efficient and reliable?

- **Task vectors via distributional alignment NeurIPS'26**
- How to correctly report LLM-as-a-judge evaluations **ICML'26**
- Adaptive hallucination detection for RAG **NeurIPS'26**

Ongoing: theory of LLM reasoning

Pillar 4 · Application mental healthcare (LLM alignment for CBT) · finance (RAG & evaluation) · vision-language-action models for robotics

Background · Few-shot In-Context Learning (ICL)

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  cheese =>                           ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← examples
3  peppermint => menthe poivrée        ←
4  plush girafe => girafe peluche      ←
5  cheese =>                           ← prompt
```

- Prepend k demonstrations (input–answer pairs) → solve a new task **without any weight update**
- More demonstrations, better accuracy — but a **longer prompt** at every query

Brown, T. B., et al. (2020). Language models are few-shot learners. *NeurIPS 33*, 1877–1901.

Today · Two complementary views of ICL

Each study **understands** ICL and turns it into **practice** — in a different space, with a different tool

PART 1

Weight space

theory · closed-form analysis

Understand: where ICL lives in attention, and **why fine-tuning can erase it**

Guide: **which parameters to fine-tune** (value only)

Lee, Sohn, Lee · ICML 2026

PART 2

Activation space

experiments · large language models

Understand: **what makes a task vector work** (d_{NTP} predicts accuracy)

Algorithm: **LTV**, a closed-form task vector

Kwon, Choi, Sohn · NeurIPS 2026

Two independent studies, one subject: ICL-level adaptation without the long prompt

PART 1 · WEIGHT SPACE (THEORY → GUIDE)

Fine-Tuning Without Forgetting ICL

"Fine-Tuning Without Forgetting In-Context Learning:
A Theoretical Analysis of Linear Attention Models"

Chungpa Lee, Jy-yong Sohn, Kangwook Lee · ICML 2026 · arXiv:2602.23197

Motivation · Fine-tuning can hurt ICL

- **ICL** adapts through few-shot prompts — flexible, but long prompts are costly
- **Fine-tuning** puts the task into the weights — strong, cheap **zero-shot** inference
- Empirically, improving zero-shot accuracy by fine-tuning can **degrade ICL**
  - this limits the fine-tuned model on tasks **not seen** during fine-tuning

Questions. How does fine-tuning change attention parameters?
When does it break ICL — and which updates keep ICL intact?

Wang et al. (2024), Loss of context-awareness; Wang et al. (2024), Two-stage LLM fine-tuning (ProMoT); Mosbach et al. (2023); Lampinen et al. (2025).

Setup 1/2 · In-context linear regression

A task = a vector $\theta \in \mathbb{R}^d$

$$x \sim \mathcal{N}(0, \Sigma)$$
$$y = \theta^\top x + e, \quad e \sim \mathcal{N}(0, \sigma^2)$$

- **Pretraining**: many random tasks θ
- **Fine-tuning**: one target task θ_0

Prompt: n demos + a query x

$$Z = \begin{bmatrix} 1 & \cdots & 1 & 1 \\ x_1 & \cdots & x_n & x \\ y_1 & \cdots & y_n & 0 \end{bmatrix}$$

- $n = 0$: **zero-shot** · $n \geq 1$: **few-shot**
- demos and query share the **same task** θ

Test error $\mathbb{E}\left[(\hat{y} - y)^2\right] \;\geq\; \sigma^2$ = noise level = best possible error

Setup 2/2 · One linear-attention layer

$$f(Z) = Z + \tfrac{1}{n+1} V Z Z^\top Q Z, \qquad \hat{y} = \text{bottom-right entry of } f(Z)$$

V: value · Q: query–key (merged), blocks ordered $(1, x, y)$:

$$V = \begin{bmatrix} 0 & \mathbf{0}_d^\top & 0 \\ \mathbf{0}_d & V_{11} & v_{12} \\ 0 & v_{21}^\top & v_{22} \end{bmatrix}, \qquad Q = \begin{bmatrix} q & \mathbf{0}_d^\top & 0 \\ \mathbf{0}_d & Q_{11} & q_{12} \\ 0 & q_{21}^\top & q_{22} \end{bmatrix}$$

- $Q_{11} \in \mathbb{R}^{d \times d}$ **learns from the demos**: weighs demo i by its match with the query, $x_i^\top Q_{11}\, x$
- $v_{21} \in \mathbb{R}^d$ **answers from memory**: a fixed predictor $v_{21}^\top x$, used even with no demos

Pretrain: few-shot error (m demos), averaged over random tasks θ

Fine-tune: **zero-shot** error on target θ_0

q: weight on the constant row of 1s (lets the model predict with no demos)

Linear attention as a tractable proxy: von Oswald et al. (2023); Ahn et al. (2023); Zhang et al. (2024)

What we show · four ways to train, in closed form

Regime	Zero-shot on θ_0	Few-shot on θ_0	Few-shot on $-\theta_0$	Message
① Pretraining m-shot loss, all tasks	worst	best	best	ICL works, but **no zero-shot** specialization
② Full FT update V, Q · zero-shot loss	best	worst	worst	Zero-shot solved, **ICL destroyed** (even on θ_0)
③ Value-only FT freeze Q · zero-shot loss	near-best	near-best	near-best	**Zero-shot gain, ICL kept** on every task
④ Value-only FT + few-shot loss ③ + few-shot loss on θ_0	near-best	best	worse	Target few-shot perfect, **costs other tasks**

Cells: test error at the optimum of each regime · $-\theta_0$ = an unseen task

Exact closed-form errors for every cell: paper Table 1

Key lens · Read the prediction off the model

After pretraining ($Q_{11} \approx \Sigma^{-1}$), the prediction with n demos is

$$\hat{y}_n \approx \underbrace{v_{21}^\top x}_{\text{answer from memory}} + \underbrace{\frac{1}{n}\sum_{i=1}^{n} y_i \left(x_i^\top Q_{11}\, x\right)}_{\text{learn from demos: weigh demo } i \text{ by its match with } x}$$

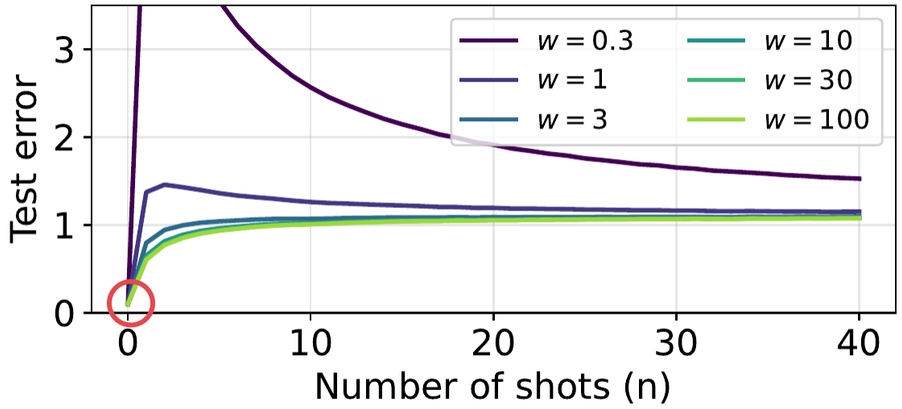
Meaning. Q_{11} learns the task from the demos (ICL); v_{21} answers from memory.

$\approx$: holds with many demos and long pretraining prompts (Ahn et al., 2023); exact form in the paper

Result 1 · Full fine-tuning breaks ICL

[Thm 1] (Full FT) Minimize the zero-shot loss over **all** of V, Q. Then the optimum has $Q_{11} = 0$ ($v_{21} = w\,\theta_0,\ q = 1/w$, any $w > 0$), and

- zero-shot error $= \sigma^2$ (optimal)
- few-shot error $>$ zero-shot error for **every** $n \geq 1$, even on θ_0



Each curve = one optimum (one w).
○ zero-shot error ($n = 0$): all at $\sigma^2 = 0.1$ (best)
$n \geq 1$: every curve jumps above it.

Meaning. Not a task mismatch: fine-tuning **removes the ICL engine** itself.

Thm 4.3, Cor 4.4 in the paper.

Proof idea of Thm 1 · why $Q_{11} \to 0$

With no demos, the query attends only to itself ($z = [1; x; 0]$):

$$\hat{y}_0 = \underbrace{q\, v_{21}^\top x}_{\text{linear in } x} + \underbrace{(v_{21}^\top x)\,(x^\top Q_{11}\, x)}_{\text{cubic in } x}$$

Zero-shot error σ^2 needs $\hat{y}_0 = \theta_0^\top x$ **exactly**:

$q\, v_{21} = \theta_0$ and $Q_{11} = 0$

Then every demo gets the same score q: a blind average, so $\hat{y}_n \to 0$ as $n \to \infty$.

Meaning. The zero-shot loss **itself** switches off the ICL engine.

Result 2 · Value-only fine-tuning preserves ICL

[Thm 2] (Value-only FT) Freeze Q at its pretrained value and train only V. The optimal value block is $v_{21} = \frac{1}{d+4}\,\theta_0$.

Full FT (Thm 1)		**Value-only (Thm 2)**	
zero-shot	σ^2	zero-shot	$\sigma^2 + \frac{2}{d+4}\,S$
few-shot, $n \to \infty$	$\sigma^2 + \|\theta\|_\Sigma^2$ (worst)	few-shot, $n \to \infty$	$\sigma^2 + \frac{1}{(d+4)^2}\,S$

Meaning. The loss is a small **fraction of the signal** S: $\frac{2}{d+4}$ (zero-shot), $\frac{1}{(d+4)^2}$ (few-shot, **every** task).

$S := \theta_0^\top \Sigma \theta_0 = \mathrm{Var}(\theta_0^\top x)$: signal size of the target task θ_0 (fixed when comparing d; $S = 1$ in the experiments)
$\|\theta\|_\Sigma^2 := \theta^\top \Sigma \theta = \mathrm{Var}(\theta^\top x)$: signal size of the evaluated task θ ($\sigma^2 + \|\theta\|_\Sigma^2$ = error of always predicting 0)

Few-shot column: $n \to \infty$, on any task θ, with the smallest change from pretraining

Thm 4.6, Prop 4.7 in the paper

Result 3 · An extra few-shot loss is not free

[Thm 3] (Value-only FT + few-shot loss on θ_0) The extra loss retunes the **weight of the ICL term** so that few-shot on θ_0 becomes exact. On another task θ, the excess error grows like $\big(1-\cos(\theta,\theta_0)\big)^2$.

Few-shot error with many demos ($n \to \infty$):

On the target θ_0

③ value-only	$\sigma^2 + \frac{1}{(d+4)^2} S$
④ + few-shot loss	$\sigma^2 \quad (\text{exact})$

On the opposite task $-\theta_0$

③ value-only	$\sigma^2 + \frac{1}{(d+4)^2} S$
④ + few-shot loss	$\sigma^2 + \frac{4}{(d+4)^2} S$

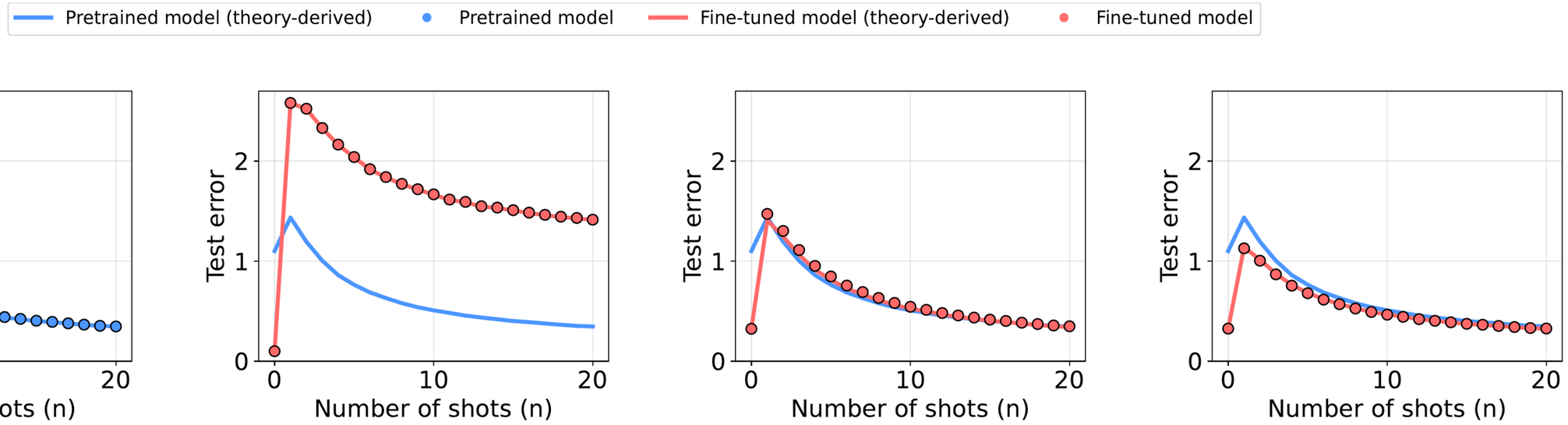
Meaning. One shared ICL weight: tuning it to θ_0 **trades away** the other tasks.

$\cos(\theta,\theta_0) = \theta^\top \Sigma \theta_0 / (\|\theta\|_\Sigma \|\theta_0\|_\Sigma)$, for tasks of equal size; $-\theta_0$ is the extreme case $\cos = -1$

Excess = extra error over the task's own best ICL weight (Thm 4.9, Prop 4.10)

Experiments · Trained linear attention matches the theory

Takeaway: trained models (dots) match the theory (lines), and ①–④ compare as predicted



① Pretraining

zero-shot worst;
improves with demos

② Full FT

zero-shot best;
demos make it **worse**

③ Value-only

near-best zero-shot;
few-shot like ①

④ Value-only + few-shot loss

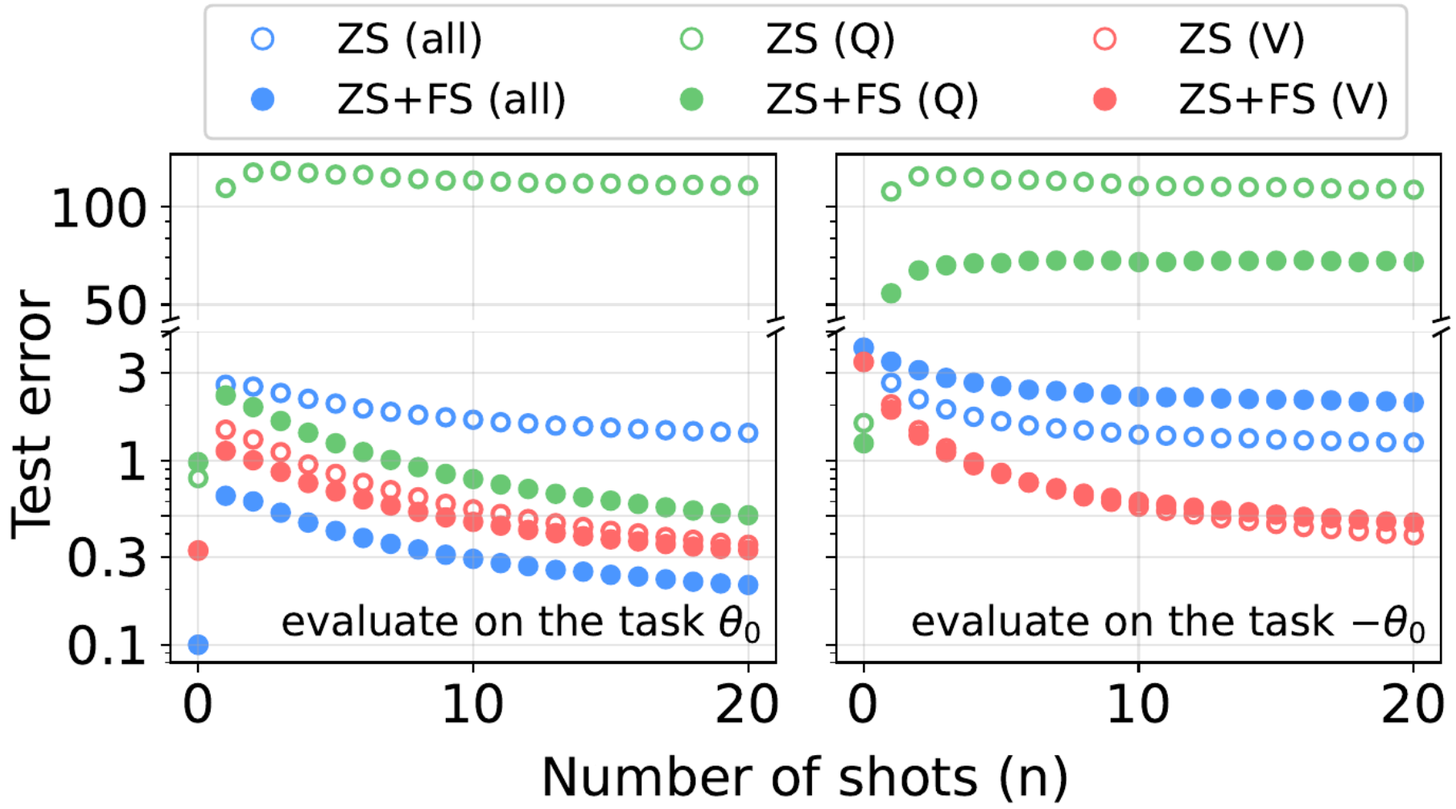
like ③, lower error
at small n on θ_0

Setting: n-shot error on $\theta_0 \cdot d = 5,\ \sigma^2 = 0.1$ · pretraining with $m = 20$ demos

Same trends with separate query/key matrices and with two heads

Experiments · In-distribution vs. an unseen task

Takeaway: on the unseen task $-\theta_0$, **value-only** keeps few-shot error lowest; updating Q hurts most



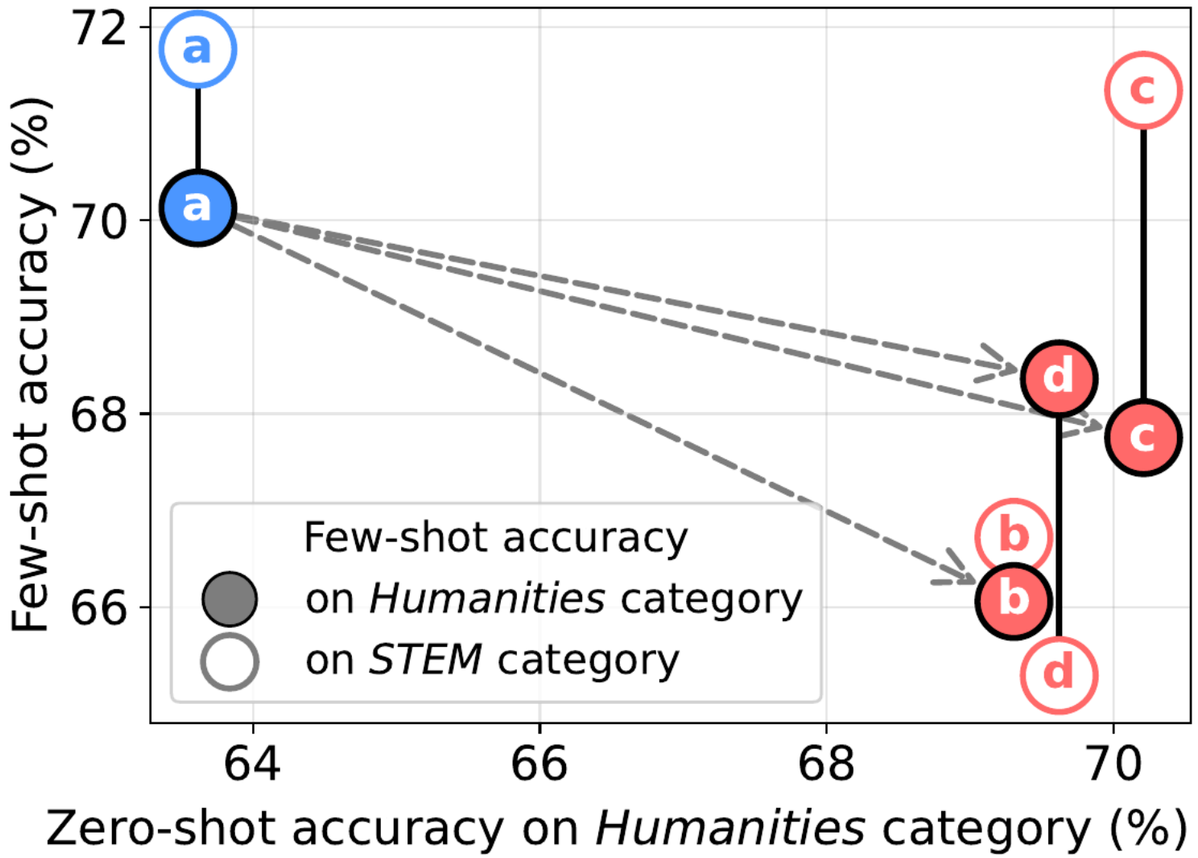
Left: target θ_0 · right: unseen $-\theta_0$

Legend: ZS = zero-shot loss only · ZS+FS = zero-shot + few-shot loss

Colors: which parameters are updated (all / Q / V)

Experiments · Qwen2.5-3B-Instruct on MMLU

Takeaway: value-only FT gets the zero-shot gain of full FT while **losing less few-shot accuracy**



- **(a)** pretrained
- **(b)** full FT
- **(c)** value-only
- **(d)** value-only + few-shot loss

x: Humanities zero-shot · y: 7-shot accuracy

filled = Humanities, open = STEM

Fine-tuned for zero-shot on Humanities (LoRA)

One model: qualitative agreement, illustrative rather than conclusive

Part 1 · Takeaways

- Fine-tuning **all** attention parameters for zero-shot accuracy can **destroy ICL**
- ICL is carried by the **query–key** matrix; task knowledge can live in the **value** matrix
- **Value-only fine-tuning**: better zero-shot, ICL preserved across tasks
- An auxiliary few-shot loss helps the target task, at a cost on dissimilar tasks

PART 2 · ACTIVATION SPACE (EXPERIMENTS → ALGORITHM)

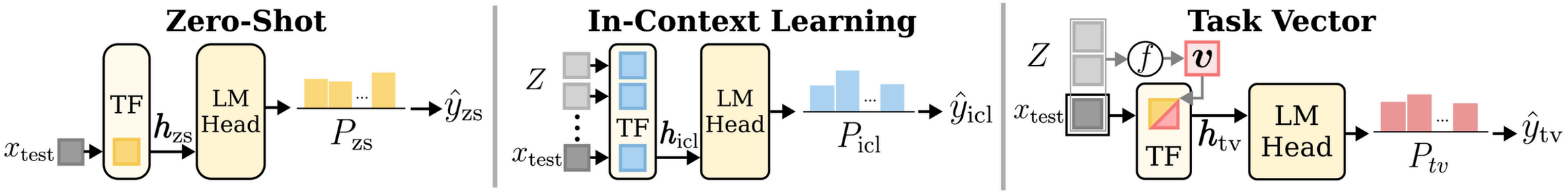
Designing Task Vectors

"Distributional Alignment as a Criterion for Designing Task Vectors in In-Context Learning"

Jihoon Kwon, Jiwon Choi, Jy-yong Sohn · arXiv:2605.20730

To appear at NeurIPS 2026

Task vectors · ICL without the demonstrations



Zero-shot	In-context learning	Task vector
query only	prepend demos Z	add one vector to the hidden state
cheap · low accuracy	**costly · high accuracy**	**cheap · high accuracy**

$$h_{\text{tv}} := h_{\text{zs}} + v$$

the task vector v moves h_{zs} toward h_{icl}

Hendel, Geva & Globerson (2023). In-context learning creates task vectors. *Findings of EMNLP 2023*.

Problem · No criterion for a good task vector

Existing methods

how should v be extracted?

- **where** — which layer?
- **what** — which token?
- **how** — mean? weighted sum?

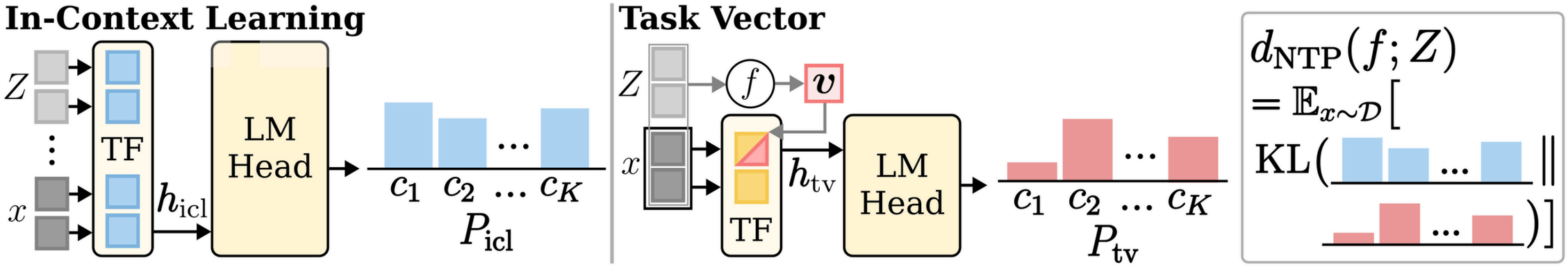
→ **hand-crafted heuristics**

As a result

- quality judged only by **downstream accuracy**
  → needs labeled queries
- no insight into **why** A beats B
  → redesign for every task and model

Design needs a principled criterion first

Criterion · Distributional alignment d_{NTP}



- $h_{\mathrm{icl}} \approx h_{\mathrm{zs}} + v = h_{\mathrm{tv}} \;\Rightarrow\; P_{\mathrm{icl}} \approx P_{\mathrm{tv}}$

  a task vector should make the model's **next-token distribution** match that of ICL

① **run with demos**
get P_{icl}

② **inject the vector**
get P_{tv}

③ **measure the gap**
between the two

④ **average**
over queries

d_{NTP} measures **how well a task vector imitates ICL**

Kwon, Choi & Sohn (2026). NeurIPS 2026. arXiv:2605.20730.

Why d_{NTP}? · A label-free quality measure

① No query labels

accuracy needs a label per query; d_{NTP} needs only unlabeled queries and the k demos ICL already uses

② Whole distribution

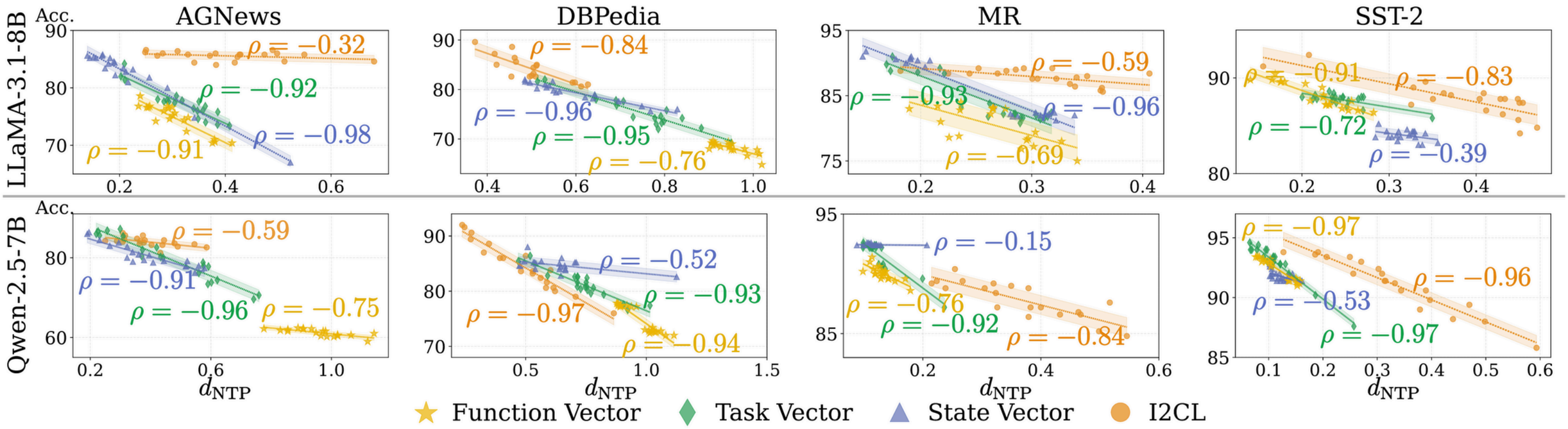
compares the **full next-token distribution**, not just the top answer

③ Compare methods

different extraction methods f can be compared **on one scale**

Quantifies the original purpose of a task vector — **“imitate ICL”**

Observation · d_{NTP} predicts accuracy



4 × 2 × 4
benchmarks × LLMs × methods

|ρ| > 0.6
negative correlation in most cells

$d_{\mathrm{NTP}} \downarrow \Rightarrow \mathrm{Acc} \uparrow$
a valid performance proxy

So — design an extraction method that directly minimizes d_{NTP}

Method · Linear Task Vector (LTV)

① The two distributions differ only through one hidden state

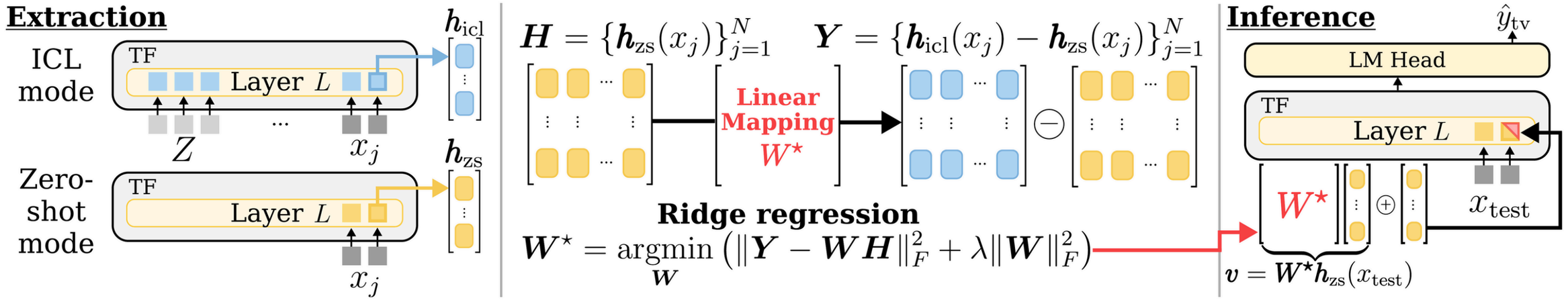
$P_\star = \mathrm{softmax}\big(g(h_\star)\big)$ with the **same** LM head g: P_{icl} and P_{tv} differ only because h_{icl} and h_{tv} differ

② If $v \approx h_{\mathrm{icl}} - h_{\mathrm{zs}}$, then d_{NTP} is small

since $h_{\mathrm{tv}} = h_{\mathrm{zs}} + v$, matching h_{tv} to h_{icl} means extracting such a v

But at test time we do **not** want to compute h_{icl}.
How can we estimate $h_{\mathrm{icl}} - h_{\mathrm{zs}}$ cheaply? → **linear regression**

LTV · A three-step pipeline



① Collect · once

N unlabeled queries, two passes each

$H = \{h_{\text{zs}}\},\ Y = \{h_{\text{icl}} - h_{\text{zs}}\}$

② Closed form · once

ridge regression, no iterative training:

$(HH^\top + \lambda I)\, W^{\star\top} = HY^\top$

③ Inject · per query

$v = W^\star h_{\text{zs}}(x)$

one matrix–vector product

A **query-specific** vector, no weight update, no demos at test time

Results ① Classification accuracy

Models	Methods	AGNews	DBPedia	HateSpeech18	MR	SST-2	SST-5	Subj	TREC	Avg.
LLaMA-2-13B [42]	Zero-shot	73.80	76.20	55.20	61.20	66.60	20.80	49.80	69.60	59.15
	ICL	88.04 (±1.7)	96.88 (±1.1)	77.40 (±2.2)	94.44 (±0.5)	94.80 (±0.7)	46.52 (±2.7)	82.28 (±7.1)	83.36 (±5.2)	82.97
	Function Vector [41]	74.00 (±1.2)	76.40 (±7.5)	54.08 (±0.4)	61.84 (±3.4)	72.44 (±6.0)	21.40 (±1.2)	50.00 (±0.0)	69.96 (±1.1)	60.02
	Task Vector [16]	79.84 (±2.9)	80.72 (±1.9)	71.48 (±6.7)	83.16 (±0.8)	82.80 (±4.2)	33.64 (±2.1)	49.72 (±0.1)	73.84 (±5.3)	69.40
	State Vector [24]	84.64 (±4.0)	89.48 (±4.9)	58.80 (±8.8)	89.20 (±2.8)	87.20 (±3.6)	36.08 (±3.0)	49.40 (±0.6)	66.96 (±1.0)	70.22
	I2CL [28]	78.88 (±0.3)	79.00 (±0.5)	54.48 (±0.1)	60.68 (±0.3)	64.80 (±0.4)	25.96 (±0.1)	50.00 (±0.0)	69.12 (±0.4)	60.37
	***LTV* (Ours)**	**86.68** (±1.7)	**93.20** (±0.5)	**72.08** (±2.2)	**90.20** (±2.0)	**88.96** (±2.1)	**41.88** (±2.2)	**78.76** (±2.7)	**83.92** (±6.3)	**79.46**
LLaMA-3.1-8B [11]	Zero-shot	75.00	69.00	60.60	82.20	86.80	25.60	59.20	49.40	63.48
	ICL	87.16 (±1.2)	97.68 (±0.8)	74.32 (±8.1)	94.52 (±0.4)	94.20 (±1.1)	48.32 (±1.1)	85.96 (±5.3)	79.08 (±7.3)	82.66
	Function Vector [41]	76.16 (±0.1)	69.44 (±1.0)	63.00 (±0.2)	83.24 (±0.3)	87.32 (±0.6)	26.20 (±1.3)	59.52 (±0.2)	69.12 (±0.6)	66.75
	Task Vector [16]	81.36 (±3.7)	83.24 (±1.5)	67.64 (±2.0)	83.48 (±3.5)	87.88 (±0.3)	34.76 (±1.9)	61.12 (±1.4)	66.48 (±1.0)	70.75
	State Vector [24]	80.28 (±4.3)	80.80 (±1.5)	65.40 (±1.3)	85.96 (±4.9)	84.12 (±0.6)	36.60 (±1.6)	62.52 (±3.5)	67.28 (±1.9)	70.37
	I2CL [28]	76.76 (±0.4)	72.56 (±0.4)	62.24 (±0.5)	85.24 (±0.3)	**90.80** (±0.2)	32.48 (±0.4)	62.28 (±0.1)	49.00 (±0.5)	66.42
	***LTV* (Ours)**	**82.84** (±2.5)	**93.36** (±1.4)	**70.44** (±6.0)	**88.68** (±1.0)	90.08 (±1.2)	**38.20** (±3.4)	**67.16** (±11.4)	**72.88** (±7.2)	**75.46**

+9.2%p

avg. accuracy vs. best baseline (State Vector, LLaMA-2-13B)

8 × 5

classification benchmarks × LLMs, consistently best

regression

also outperforms baselines on regression tasks

Closes much of the gap to ICL — **at zero-shot cost**

Results ② Inference cost

	Zero-shot	ICL	Function Vector	I2CL	State Vector	Task Vector	***LTV* (Ours)**
Extraction (s) ↓	-	-	198.965	**1.347**	27.756	27.773	42.664
Inference (s) ↓	0.0220	0.2238	0.0614	0.0246	0.0292	0.0236	**0.0226**
Accuracy (%) ↑	63.48	82.66	66.75	66.42	70.37	70.75	**75.46**

Per-query inference **0.0226 s** — zero-shot level (0.0220 s), ~1/10 of ICL (0.2238 s)

The 42.7 s extraction cost is paid once for the whole test set

Results ③ Transfer across model scales

Extract the vector from a large model, inject it into a small one

Models	Methods	AGNews	DBPedia	HateSpeech18	MR	SST-2	SST-5	Subj	TREC	Avg.
Small Model (*Qwen-2.5-7B* [38])	Zero-shot	74.20	73.20	61.20	62.20	55.40	20.00	56.80	67.20	58.78
	ICL	83.24	83.48	80.60	92.88	95.00	46.60	76.00	87.04	80.60
	LTV	78.48	77.24	72.28	87.56	89.00	33.16	66.52	84.32	73.57
Large Model (*Qwen-2.5-72B* [38])	Zero-shot	72.00	68.00	70.40	92.00	90.60	36.00	53.00	73.00	69.38
	ICL	88.08	95.64	82.80	93.88	96.44	46.12	95.80	75.44	84.25
	LTV	81.40	88.92	77.20	91.48	94.36	37.16	91.92	77.24	79.96
Transferred *LTV* (72B → 7B)		**83.24** (+4.76)	**92.68** (+15.44)	**76.48** (+4.20)	**91.36** (+3.80)	**92.36** (+3.36)	**42.48** (+9.32)	**90.24** (+23.72)	71.16 (-13.16)	**80.00** (+6.43)

72B → 7B transfer: **+6.4%** for the smaller model

A new use of extracted task representations

Conclusion · Two views of ICL

Weight space · ICML 2026

- full fine-tuning can erase ICL
- query–key = ICL mechanism; value = task knowledge
- **value-only FT** keeps ICL while improving zero-shot

Activation space · NeurIPS 2026

- good task vector = **matches ICL's next-token distribution** (d_{NTP})
- **LTV**: closed-form, query-specific, zero-shot cost
- transfers from large to small models

Theory → a fine-tuning guide · Experiments → an algorithm

both aim at ICL-level adaptation without the long prompt

Thank you

arXiv:2602.23197 (ICML 2026) · arXiv:2605.20730 (NeurIPS 2026)

Jy-yong Sohn · Yonsei University
ITML Lab · itml.yonsei.ac.kr